

Inhaltsverzeichnis

Numerische Simulation eines Bipolartransistors

Arbeitspunktberechnung mit Newton-Raphson und Bisektionsverfahren

Begleitmaterial zur Lehrveranstaltung

Elektronik / Numerische Methoden in der Elektrotechnik

Programm: Transistor_20.py

Sprache: Python 3 | Bibliotheken: NumPy, Matplotlib

1 Einleitung und Zielsetzung

Dieses Skript erläutert Schritt für Schritt das Programm Transistor_20.py. Das Programm berechnet den Arbeitspunkt eines NPN-Bipolartransistors in einer einfachen Kollektorschaltung - und zwar vollständig numerisch, ohne Vereinfachungen wie die oft verwendete Beta-Näherung.

Ziel des Programms: Den Basiswiderstand R_B so bestimmen, dass der Transistor im Mittelpunkt seiner Aussteuerbarkeit arbeitet, d. h. $V_{CE} \approx V_{CC} / 2$.

Dazu werden zwei numerische Verfahren kombiniert:

- **Newton-Raphson-Verfahren** (innere Schleife): Löst das nichtlineare Gleichungssystem für einen festen Wert von R_B .
- **Bisektionsverfahren** (äußere Schleife): Sucht iterativ den optimalen R_B -Wert.

2 Physikalischer Hintergrund

2.1 Die Transistorschaltung

Betrachtet wird ein NPN-Bipolartransistor mit Basiswiderstand R_B und Kollektorwiderstand R_C , gespeist von der Versorgungsspannung V_{CC} :

Schaltungsaufbau (vereinfacht)

```
V_CC
|
[R_C] <- Kollektorwiderstand
|
+---[C] NPN-Transistor
|
[R_B] <- Basiswiderstand
|
V_BB (= V_CC)
|
GND (Emitter)
```

Die Knoten- und Maschengleichungen dieser Schaltung liefern:

$$I_B = (V_{BB} - V_{BE}) / R_B \text{ [Maschengleichung Basis]}$$

$$I_C = (V_{CC} - V_{CE}) / R_C \text{ [Maschengleichung Kollektor]}$$

2.2 Das Shockley-Diodenmodell

Der Basis-Emitter-Übergang verhält sich wie eine Diode. Der Basisstrom folgt der Shockley-Gleichung:

$$I_B = I_S \cdot \exp(q \cdot V_{BE} / (k_B \cdot T))$$

mit den Konstanten:

Parameter	Wert	Bedeutung
I_S	$1 \cdot 10^{-13} \text{ A}$	Sättigungssperrstrom des Transistors
q	$1,602 \cdot 10^{-19} \text{ C}$	Elementarladung
k_B	$1,381 \cdot 10^{-23} \text{ J/K}$	Boltzmann-Konstante
T	300 K	Absolute Temperatur (Raumtemperatur)

Die Größe $V_T = k_B \cdot T / q \approx 25,85 \text{ mV}$ bei 300 K wird als Temperaturspannung bezeichnet und taucht in allen Dioden- und Transistorgleichungen auf.

2.3 Das Ebers-Moll-Modell mit Early-Effekt

Im realen Transistor hängt der Kollektorstrom nicht nur vom Basisstrom, sondern auch von der Kollektor-Emitter-Spannung V_{CE} ab. Diesen Effekt beschreibt der Early-Effekt:

$$I_C = I_B \cdot (1 + V_{CE} / V_A)$$

Die **Early-Spannung** V_A (hier 100 V) charakterisiert, wie stark der Kollektorstrom mit V_{CE} ansteigt. Ein großes V_A bedeutet einen nahezu idealen, spannungsunabhängigen Transistor.

3 Das nichtlineare Gleichungssystem

Der Arbeitspunkt des Transistors ist derjenige Betriebszustand, in dem alle Strom- und Spannungsbedingungen gleichzeitig erfüllt sind. Dies führt auf ein System von zwei nichtlinearen Gleichungen in den Unbekannten V_{BE} und V_{CE} :

$$\text{eq1} = (V_{BB} - V_{BE})/R_B - I_S \cdot \exp(q \cdot V_{BE} / (k_B \cdot T)) = 0$$

$$\text{eq2} = (V_{CC} - V_{CE})/R_C - I_S \cdot \exp(q \cdot V_{BE} / (k_B \cdot T)) \cdot (1 + V_{CE}/V_A) = 0$$

Gleichung 1 setzt die Maschengleichung für den Basiskreis gleich dem Shockley-Ausdruck.

Gleichung 2 setzt die Kollektormaschengleichung gleich dem Ebers-Moll-Ausdruck.

Da $\exp(\cdot)$ eine stark nichtlineare Funktion ist, kann dieses System nicht analytisch gelöst werden – hier kommt das Newton-Raphson-Verfahren ins Spiel.

4 Das Newton-Raphson-Verfahren

4.1 Wozu brauchen wir das Verfahren?

In Kapitel 3 haben wir das Gleichungssystem aufgestellt:

$$\text{eq1}(V_{BE}, V_{CE}) = 0$$

$$\text{eq2}(V_{BE}, V_{CE}) = 0$$

Beide Gleichungen enthalten den Term $\exp(q \cdot V_{BE} / k_B \cdot T)$. Die Exponentialfunktion macht eine analytische (exakte) Lösung nach V_{BE} und V_{CE} unmöglich – es gibt keine algebraische Umformung, die zu einer geschlossenen Formel führt.

Wir brauchen daher ein numerisches Verfahren, das iterativ eine Näherungslösung berechnet. Das Newton-Raphson-Verfahren ist hierfür besonders geeignet, weil es sehr schnell konvergiert (quadratisch, s. Abschnitt 4.6) und weil wir die Ableitungen analytisch kennen.

4.2 Grundidee: Linearisierung um den aktuellen Punkt

Die Idee ist einfach: Eine nichtlineare Funktion lässt sich in der Nähe eines Punktes durch ihre Tangente annähern. Das ist die Taylor-Entwicklung erster Ordnung.

Für eine einzelne Funktion $f(x)$ gilt in der Nähe von x_n :

$$f(x) \approx f(x_n) + f'(x_n) \cdot (x - x_n)$$

Wir suchen die Nullstelle, also $f(x) = 0$. Einsetzen und nach x auflösen:

$$0 = f(x_n) + f'(x_n) \cdot (x - x_n)$$

$$\mathbf{x} - \mathbf{x}_n = -\mathbf{f}(\mathbf{x}_n) / \mathbf{f}'(\mathbf{x}_n)$$

$$\mathbf{x}_{\{n+1\}} = \mathbf{x}_n - \mathbf{f}(\mathbf{x}_n) / \mathbf{f}'(\mathbf{x}_n) \text{ [1D Newton-Formel]}$$

Das ist der klassische Newton-Schritt für eine einzelne Gleichung. Wir haben aber zwei Gleichungen und zwei Unbekannte – wir müssen auf ein Vektorsystem verallgemeinern.

4.3 Verallgemeinerung auf Systeme: Die Jacobi-Matrix

Unser Zustandsvektor ist:

$$\mathbf{x} = [\mathbf{V_BE}, \mathbf{V_CE}]^T \text{ (ein Spaltenvektor mit 2 Einträgen)}$$

Unser Fehlervektor ist:

$$\mathbf{F}(\mathbf{x}) = [\text{eq1}(\mathbf{V_BE}, \mathbf{V_CE}), \text{eq2}(\mathbf{V_BE}, \mathbf{V_CE})]^T$$

Die Taylor-Entwicklung erster Ordnung für ein Vektorsystem lautet:

$$\mathbf{F}(\mathbf{x}) \approx \mathbf{F}(\mathbf{x}_n) + \mathbf{J}(\mathbf{x}_n) \cdot (\mathbf{x} - \mathbf{x}_n)$$

Dabei ist J die Jacobi-Matrix – das mehrdimensionale Analogon zur Ableitung. Sie enthält alle partiellen Ableitungen von F nach den Komponenten von x:

$$\mathbf{J} = \begin{vmatrix} \partial \text{eq1} / \partial \mathbf{V_BE} & \partial \text{eq1} / \partial \mathbf{V_CE} \\ \partial \text{eq2} / \partial \mathbf{V_BE} & \partial \text{eq2} / \partial \mathbf{V_CE} \end{vmatrix}$$

Jeder Eintrag $J[i][j]$ beantwortet die Frage: "Wenn ich die j-te Variable ($\mathbf{V_BE}$ oder $\mathbf{V_CE}$) ein kleines Stück verändere – wie stark ändert sich die i-te Gleichung (eq1 oder eq2)?"

Nullstelle der linearisierten Gleichung suchen, also $\mathbf{F}(\mathbf{x}) = 0$ setzen:

$$\mathbf{0} = \mathbf{F}(\mathbf{x}_n) + \mathbf{J}(\mathbf{x}_n) \cdot \Delta \mathbf{x}$$

$$\mathbf{J}(\mathbf{x}_n) \cdot \Delta \mathbf{x} = -\mathbf{F}(\mathbf{x}_n) \text{ [Das lineare Gleichungssystem]}$$

$$\mathbf{x}_{\{n+1\}} = \mathbf{x}_n + \Delta \mathbf{x} \text{ [Aktualisierungsschritt]}$$

Das ist die Newton-Raphson-Formel für Systeme. Das Symbol $\Delta \mathbf{x}$ bezeichnet den Korrekturvektor $[\Delta \mathbf{V_BE}, \Delta \mathbf{V_CE}]^T$.

4.4 Herleitung der partiellen Ableitungen (Jacobi-Einträge)

Jetzt leiten wir jeden der vier Jacobi-Einträge Schritt für Schritt her. Zur Erinnerung die beiden Gleichungen:

$$\text{eq1} = (\mathbf{V_BB} - \mathbf{V_BE}) / \mathbf{R_B} - \mathbf{I_S} \cdot \exp(\mathbf{q} \cdot \mathbf{V_BE} / (\mathbf{k_B} \cdot \mathbf{T}))$$

$$\text{eq2} = (\mathbf{V_CC} - \mathbf{V_CE}) / \mathbf{R_C} - \mathbf{I_S} \cdot \exp(\mathbf{q} \cdot \mathbf{V_BE} / (\mathbf{k_B} \cdot \mathbf{T})) \cdot (1 + \mathbf{V_CE} / \mathbf{V_A})$$

Zur Abkürzung definieren wir zwei Hilfsgrößen, die im Programm ebenfalls verwendet werden:

$$\text{exp_term} = \exp(\mathbf{q} \cdot \mathbf{V_BE} / (\mathbf{k_B} \cdot \mathbf{T}))$$

$$\text{factor} = \mathbf{I_S} \cdot (\mathbf{q} / (\mathbf{k_B} \cdot \mathbf{T})) \cdot \text{exp_term} \text{ [= Ableitung des Shockley-Terms]}$$

4.4.1 Ableitung $\partial \text{eq1} / \partial \mathbf{V_BE}$

eq1 besteht aus zwei Termen. Wir leiten beide nach $\mathbf{V_BE}$ ab:

$$\mathbf{d/dV_BE} [(\mathbf{V_BB} - \mathbf{V_BE}) / \mathbf{R_B}] = -1 / \mathbf{R_B}$$

Der zweite Term ist eine Exponentialfunktion. Mit der Kettenregel gilt:

$$\mathbf{d/dV_BE} [\mathbf{I_S} \cdot \exp(\mathbf{q} \cdot \mathbf{V_BE} / (\mathbf{k_B} \cdot \mathbf{T}))] = \mathbf{I_S} \cdot (\mathbf{q} / (\mathbf{k_B} \cdot \mathbf{T})) \cdot \exp(\dots) = \text{factor}$$

Da eq1 das Minus-Zeichen trägt, ergibt sich insgesamt:

$$\partial \text{eq1} / \partial \mathbf{V_BE} = -1 / \mathbf{R_B} - \text{factor}$$

Im Programm steht genau das:

$$d_eq1_dV_BE = -1/R_B - factor$$

4.4.2 Ableitung $\partial eq1 / \partial V_CE$

In eq1 taucht V_CE überhaupt nicht auf – weder im Maschenterms noch im Shockley-Ausdruck. Daher ist die partielle Ableitung exakt null:

$$\partial eq1 / \partial V_CE = 0$$

Das bedeutet: Die erste Gleichung (Basiskreis) ist vollständig unabhängig von V_CE . Im Programm:

$$d_eq1_dV_CE = 0.0$$

4.4.3 Ableitung $\partial eq2 / \partial V_BE$

eq2 enthält V_BE nur im Exponentialterm. Der Maschen-Term $(V_CC - V_CE)/R_C$ ist konstant bezüglich V_BE . Wir leiten nur den Ebers-Moll-Ausdruck ab:

$$d/dV_BE [I_S \cdot \exp(q \cdot V_BE / (k_B \cdot T)) \cdot (1 + V_CE/V_A)]$$

Der Faktor $(1 + V_CE/V_A)$ ist bezüglich V_BE eine Konstante. Mit der Kettenregel:

$$= I_S \cdot (q / (k_B \cdot T)) \cdot \exp(...) \cdot (1 + V_CE/V_A) = factor \cdot (1 + V_CE/V_A)$$

Da eq2 auch hier ein Minuszeichen trägt:

$$\partial eq2 / \partial V_BE = -factor \cdot (1 + V_CE/V_A)$$

Im Programm:

$$d_eq2_dV_BE = -factor * (1 + V_CE / V_A)$$

4.4.4 Ableitung $\partial eq2 / \partial V_CE$

eq2 enthält V_CE an zwei Stellen: im Maschen-Term und im Early-Faktor. Wir leiten beide ab:

$$d/dV_CE [(V_CC - V_CE)/R_C] = -1/R_C$$

Für den Ebers-Moll-Term gilt – $\exp_term$ ist bezüglich V_CE eine Konstante:

$$d/dV_CE [I_S \cdot \exp_term \cdot (1 + V_CE/V_A)] = I_S \cdot \exp_term \cdot (1/V_A) = I_S/V_A \cdot \exp_term$$

Da eq2 das Minuszeichen trägt, wird dieser Term positiv subtrahiert:

$$\partial eq2 / \partial V_CE = -1/R_C + (I_S/V_A) \cdot \exp_term$$

Im Programm:

$$d_eq2_dV_CE = -1/R_C + (I_S / V_A) * \exp_term$$

4.4.5 Die vollständige Jacobi-Matrix

Alle vier Ableitungen zusammengesetzt ergeben:

$$J = \begin{vmatrix} -1/R_B - factor & 0 \\ -factor \cdot (1 + V_CE/V_A) & -1/R_C + I_S/V_A \cdot \exp_term \end{vmatrix}$$

Diese 2×2-Matrix wird in jedem Newton-Schritt neu berechnet, weil sich $\exp_term$ und $factor$ mit jedem neuen Schätzwert für V_BE ändern.

```
def J(vars):
```

```
    V_BE, V_CE = vars
```

```
    exp_term = np.exp(q * V_BE / (k_B * T)) # e^(q*V_BE/k_B*T)
```

```
    factor = I_S * q / (k_B * T) * exp_term # Ableitung Shockley
```

```
    d_eq1_dV_BE = -1/R_B - factor # s. 4.4.1
```

```

d_eq1_dV_CE = 0.0 # s. 4.4.2
d_eq2_dV_BE = -factor * (1 + V_CE / V_A) # s. 4.4.3
d_eq2_dV_CE = -1/R_C + (I_S / V_A) * exp_term # s. 4.4.4
return np.array([[d_eq1_dV_BE, d_eq1_dV_CE],
[d_eq2_dV_BE, d_eq2_dV_CE]])

```

4.5 Das lineare Gleichungssystem lösen - warum nicht J^{-1} berechnen?

Die Newton-Formel lautet:

$$\Delta \mathbf{x} = -\mathbf{J}(\mathbf{x}_n)^{-1} \cdot \mathbf{F}(\mathbf{x}_n)$$

Man könnte J invertieren und dann Δx berechnen. Das tut man in der Praxis jedoch nicht – aus zwei Gründen:

- **Numerische Stabilität:** Die direkte Matrixinversion verstärkt Rundungsfehler. Lineare Gleichungslöser (wie LU-Zerlegung) sind stabiler.
- **Rechenaufwand:** Für $n \times n$ -Systeme kostet Inversion $O(n^3)$, ein Gleichungslöser ebenfalls $O(n^3)$, aber mit kleinerem Vorfaktor.

Stattdessen löst man das äquivalente lineare System:

$$\mathbf{J}(\mathbf{x}_n) \cdot \Delta \mathbf{x} = -\mathbf{F}(\mathbf{x}_n)$$

Das entspricht der Frage: "Welche Korrektur Δx muss ich anwenden, damit $J \cdot \Delta x$ den Fehler $-F$ genau ausgleicht?" NumPy löst das mit LU-Zerlegung:

```

delta = np.linalg.solve(J_val, -f_val) # löst J·Δx = -F
xy += delta # x_{n+1} = x_n + Δx

```

4.6 Schritt-für-Schritt durch eine vollständige Iteration

Hier verfolgen wir eine Newton-Iteration mit konkreten Zahlenwerten ($R_B = 3000 \, \Omega$, $V_{CC} = 15 \, V$):

Iteration 0: Startwert

$x_0 = [V_{BE}, V_{CE}] = [0.9 \, V, 8.0 \, V]$

Berechne $F(x_0)$:

$I_B(\text{Masche}) = (15 - 0.9) / 3000 = 4.700e-3 \, A$

$I_B(\text{Shockley}) = 1e-13 \cdot \exp(0.9/0.02585) = 4.140e-3 \, A$

$eq1 = 4.700e-3 - 4.140e-3 = +5.60e-4$ (noch nicht null!)

$I_C(\text{Masche}) = (15 - 8.0) / 1000 = 7.000e-3 \, A$

$I_C(\text{Ebers}) = 4.140e-3 \cdot (1 + 8/100) = 4.471e-3 \, A$

$eq2 = 7.000e-3 - 4.471e-3 = +2.53e-3$ (noch nicht null!)

=> Fehlervektor noch weit von null entfernt => weiter iterieren

Iteration 0: Jacobi-Matrix und Korrekturschritt

Berechne $J(x_0)$:

$\exp_term = \exp(0.9/0.02585) = 4.140e10$

$factor = 1e-13 \cdot (1/0.02585) \cdot \exp_term = 1.601e-1$

$J[0][0] = -1/3000 - 0.1601 = -0.1604$

$J[0][1] = 0.0$

$J[1][0] = -0.1601 \cdot (1 + 8/100) = -0.1729$

$J[1][1] = -1/1000 + (1e-13/100) \cdot \exp = -0.0010$

Löse $J \cdot \Delta x = -F$:

$\Delta x = \text{np.linalg.solve}(J, -F) \Rightarrow [\Delta V_{BE}, \Delta V_{CE}]$

Aktualisiere:

$$V_BE_neu = 0.9 + \Delta V_BE$$

$$V_CE_neu = 8.0 + \Delta V_CE$$

=> nächste Iteration beginnt mit verbessertem Schätzwert

Nach 5-8 solcher Iterationen sind beide Fehlerkomponenten kleiner als 10^{-10} – das Verfahren hat konvergiert.

4.7 Konvergenzverhalten und Konvergenzrate

Newton-Raphson hat unter geeigneten Bedingungen quadratische Konvergenz. Das bedeutet: Wenn der Fehler in Iteration n gleich ε_n ist, dann gilt näherungsweise:

$$\varepsilon_{\{n+1\}} \approx C \cdot \varepsilon_n^2 \text{ [quadratische Konvergenz]}$$

In der Praxis sieht das so aus:

Parameter	Wert	Bedeutung
Iteration 1	Fehler $\approx 10^{-1}$	Erster grober Schritt
Iteration 2	Fehler $\approx 10^{-2}$	Deutliche Verbesserung
Iteration 3	Fehler $\approx 10^{-4}$	Fehler quadriert sich
Iteration 4	Fehler $\approx 10^{-8}$	Sehr schnelle Annäherung
Iteration 5	Fehler $\approx 10^{-16}$	Maschinengenaugkeit erreicht

Warum quadratisch? Der Taylor-Entwicklung erster Ordnung bleibt ein Restterm zweiter Ordnung übrig: $F(x) = F(x_n) + J \cdot \Delta x + O(\Delta x^2)$. Newton eliminiert exakt den linearen Term – was übrig bleibt, ist $O(\Delta x^2)$. Der nächste Fehler ist also proportional zum Quadrat des aktuellen Fehlers.

Wichtige Voraussetzung: Der Startwert muss in der Nähe der echten Lösung liegen. Weit entfernte Startwerte können zu Divergenz oder Konvergenz zu einer falschen Lösung führen. Im Programm werden deshalb physikalisch sinnvolle Startwerte [0.9 V, 8.0 V] gewählt.

4.8 Vollständiger Algorithmus im Programm (newton_method)

Alle Schritte zusammengefasst als Ablaufdiagramm:

Ablauf newton_method(initial_guess, tol=1e-10, max_iter=100)

EINGABE: Startwert [V_{BE}^0 , V_{CE}^0], Toleranz, max. Iterationen

SCHRITT 0: $x = [0.9, 8.0]$ (oder adaptiver Startwert)

SCHLEIFE ($i = 0, 1, 2, \dots$):

SCHRITT 1: Fehlervektor berechnen

$$f_val = F(x) = [eq1(x), eq2(x)]$$

SCHRITT 2: Konvergenzprüfung

WENN $|eq1| < 1e-10$ UND $|eq2| < 1e-10$:

RÜCKGABE x , Anzahl Iterationen

SCHRITT 3: Jacobi-Matrix berechnen

$$J_val = J(x) \text{ [4 partielle Ableitungen, s. Kap. 4.4]}$$

SCHRITT 4: Korrekturvektor lösen

$$\Delta x = \text{np.linalg.solve}(J_val, -f_val)$$

[löst $J \cdot \Delta x = -F$ ohne explizite Matrixinversion]

SCHRITT 5: Schätzwert aktualisieren

$$x = x + \Delta x$$

SCHRITT 6: Physikalische Plausibilität prüfen

WENN $V_{CE} < 0$: FEHLER (unphysikalische Lösung)

NACH max_iter Schritten ohne Konvergenz: FEHLER

```

def newton_method(initial_guess, tol=1e-10, max_iter=100):
    xy = np.array(initial_guess, dtype=float) # Schritt 0
    for i in range(max_iter):
        f_val = F(xy) # Schritt 1
        if np.all(np.abs(f_val) < tol): # Schritt 2
            return xy, i+1
        J_val = J(xy) # Schritt 3
        delta = np.linalg.solve(J_val, -f_val) # Schritt 4
        xy += delta # Schritt 5
        if xy[1] < 0: # Schritt 6
            raise ValueError('V_CE negativ!')
        raise ValueError('Keine Konvergenz!')

```

5 Das Bisektionsverfahren für R_B

5.1 Grundprinzip

Das Newton-Raphson-Verfahren löst das innere Problem für ein festes R_B . Das äußere Problem ist: Welcher Wert von R_B führt zu $V_{CE} = V_{CC}/2$?

Das Bisektionsverfahren nutzt aus, dass V_{CE} eine monotone Funktion von R_B ist:

- **R_B zu groß** → wenig Basisstrom → Transistor kaum leitend → $V_{CE} \approx V_{CC}$ (hoch)
- **R_B zu klein** → viel Basisstrom → Transistor stark leitend → $V_{CE} \approx 0$ (niedrig)

Es existiert also genau ein R_B , für den $V_{CE} = V_{CC}/2$ gilt.

5.2 Algorithmus

$R_{B_try} = (R_{B_min} + R_{B_max}) / 2$

1. Setze Startwerte: $R_{B_min} = 500 \Omega$, $R_{B_max} = 100 \text{ k}\Omega$
2. Berechne Mittelpunkt $R_{B_try} = (R_{B_min} + R_{B_max}) / 2$
3. Löse inneres System mit Newton-Raphson → V_{CE}
4. **Toleranzprüfung:** Falls $|V_{CE} - V_{CC}/2| \leq 5\% \cdot V_{CC}/2 \rightarrow$ Fertig!
5. **Aktualisierung:** V_{CE} zu klein → $R_{B_max} = R_{B_try}$; V_{CE} zu groß → $R_{B_min} = R_{B_try}$
6. Zurück zu Schritt 2

Nach n Bisektionsschritten hat sich das Suchintervall auf $(R_{B_max} - R_{B_min}) / 2^n$ verkleinert.
Nach 30 Schritten aus $100 \text{ k}\Omega$ Startbreite: Genauigkeit $\approx 0,1 \text{ m}\Omega$.

6 Programmstruktur im Überblick

Modularer Aufbau von Transistor_20.py

F(vars) Fehlerfunktion: gibt [eq1, eq2] zurück

J(vars) Jacobi-Matrix: gibt 2×2 -Matrix zurück

newton_method() Newton-Raphson-Solver (innere Schleife)

finde_R_B() Bisektionsoptimierer (äußere Schleife)

__main__ Hauptprogramm + Ergebnisplot

Eine Besonderheit: R_B ist im Programm als globale Variable deklariert, damit $F()$ und $J()$ den aktuellen Wert der äußeren Bisektion kennen. Dies ist funktional, in modernem Python würde man R_B besser als Parameter übergeben.

7 Ergebnisplot und Arbeitspunkt

Nach Konvergenz beider Verfahren wird ein Diagramm mit drei Elementen erstellt:

- **Transistorkennlinie $I_C(V_{CE})$:** Für den berechneten Basisstrom I_B gilt: $I_C(V_{CE}) = I_B \cdot (1 + V_{CE}/V_A)$. Dies ist eine leicht ansteigende Gerade (wegen des Early-Effekts).
- **Arbeitsgerade (Lastlinie):** $I_C = (V_{CC} - V_{CE})/R_C$. Eine fallende Gerade von $(0, V_{CC}/R_C)$ bis $(V_{CC}, 0)$.
- **Arbeitspunkt (roter Punkt):** Der Schnittpunkt beider Linien – der einzige Betriebszustand, der alle Gleichungen gleichzeitig erfüllt.

Optimaler Arbeitspunkt: Der Punkt sollte in der Mitte der Lastlinie liegen ($V_{CE} = V_{CC}/2$), um maximale unverzerrte Aussteuerung zu ermöglichen.

8 Hinweise zur Implementierung

8.1 Stärken des Programms

- Analytische Jacobi-Matrix: genauer und schneller als numerische Differentiation
- Adaptive Startwerte: Das Newton-Verfahren startet beim nächsten Bisektionsschritt mit dem letzten konvergierten Ergebnis – verbessert Stabilität
- Physikalische Plausibilitätsprüfung: $V_{CE} < 0$ wird als Fehler erkannt
- Klare Trennung: Gleichungssystem (F, J) ist unabhängig vom Solver (`newton_method`)

8.2 Verbesserungspotenzial

- **Globale Variable R_B :** Sollte als Parameter an $F()$ und $J()$ übergeben werden. Empfehlung: Closures oder Klassenstruktur verwenden.
- **Fehlende Rückgabe bei Newton-Fehler:** Wenn Newton nicht konvergiert, wird $V_{CE} = -1$ gesetzt, aber V_{BE} bleibt undefiniert – bei Schleifenabbruch würde ein `NameError` auftreten.
- **Plot-Funktion:** Ist als innere Funktion im `__main__`-Block definiert; besser auf Modulebene auslagern für Wiederverwendbarkeit.

9 Übungsaufgaben

Zur Vertiefung des Verständnisses:

7. Berechnen Sie $V_T = k_B \cdot T/q$ bei 300 K per Hand und vergleichen Sie mit dem Wert im Programm.
8. Wie verändert sich der Arbeitspunkt, wenn R_C auf 2 k Ω verdoppelt wird? Schätzen Sie ab, welches R_B dann benötigt wird.
9. Warum konvergiert Newton-Raphson schneller mit adaptiven Startwerten? Erklären Sie anhand der Konvergenzbedingung.
10. Skizzieren Sie das Bisektionsverfahren grafisch: V_{CE} als Funktion von R_B , und markieren Sie den Zielbereich.
11. Ergänzen Sie das Programm so, dass R_B nicht als globale Variable, sondern als Parameter übergeben wird (Hinweis: Lambda-Funktion oder Closure).
12. Was passiert, wenn $V_A \rightarrow \infty$ gesetzt wird? Welches Modell ergibt sich dann?

10 Zusammenfassung

Kernpunkte dieses Skripts

1. Transistorarbeitspunkt = Lösung eines nichtlinearen 2x2-Gleichungssystems
 2. Shockley-Gleichung beschreibt den exponentiellen Basisstrom
 3. Ebers-Moll + Early-Effekt beschreiben den Kollektorstrom
 4. Newton-Raphson löst das innere Problem (festes R_B) quadratisch konvergent
 5. Bisektionsverfahren optimiert R_B für den Arbeitspunkt $V_{CE} = V_{CC}/2$
 6. Analytische Jacobi-Matrix ist essenziell für Genauigkeit und Geschwindigkeit
 7. Ziel: maximale Aussteuerbarkeit $\Rightarrow V_{CE}$ liegt in der Mitte der Lastlinie
-

Ende des Skripts